CREDCHAIN

Hybrid Blockchain-Backed Digital Credential Verification & Lifecycle Management System

Academic Project Report

Project: CredChain
License: MIT

Team Structure

Member 1 — Blockchain Core & Trust Layer Engineer
Responsible for the smart-contract trust layer, institution and issuer authorization, credential lifecycle contracts, blockchain testing, and the React administrative platform.

Member 2 — Blockchain Application & Verification Engineer
Responsible for the Express backend, managed institutional transaction signing, SQLite persistence, credential verification, event synchronization, analytics, audit services, and the lightweight issuer/verifier portals.

Student Names: [Implementation detail not available in inspected project]
Institution: [Implementation detail not available in inspected project]
Department: [Implementation detail not available in inspected project]
Academic Year: 2026

---

Abstract

CredChain is a hybrid blockchain-backed digital credential verification and lifecycle management system designed to address document tampering, centralized trust weaknesses, cross-institution credential isolation, loss of historical versions, and the practical difficulty of requiring ordinary verifiers to interact directly with Web3 wallets. Rather than storing academic certificates themselves on a blockchain, CredChain stores only the cryptographic and lifecycle information required to establish trust. Personally identifiable information and application metadata remain off-chain.

The implemented system uses Solidity smart contracts on an EVM-compatible Hardhat development network to store SHA-256 credential digests, certificate identifiers, issuer addresses, institution bindings, timestamps, lifecycle state, and version history. An Express.js backend uses Ethers.js to communicate with the blockchain, performs managed institutional transaction signing, processes uploaded PDF files entirely in memory, maintains SQLite metadata and audit caches, and provides public verification without requiring a blockchain wallet. The system also provides a React-based administration interface and lightweight HTML/JavaScript issuer and public verification portals.

Three smart contracts divide responsibility between institutional identity management, certificate lifecycle management, and a facade used as the external contract entry point.

On-chain authorization ensures that credentials can be issued or modified only by wallets authorized for the relevant institution. Verification compares the SHA-256 digest of an uploaded PDF with the immutable digest represented in the current blockchain state.

The implemented prototype operates on a local Hardhat blockchain with Chain ID 31337. Thirty-three smart-contract tests are documented as passing. Public testnet deployment, decentralized document storage, zero-knowledge proofs, Layer-2 deployment, and production-grade key custody remain outside the current implementation.

---

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System Analysis
5. Proposed System
6. Advantages of the Proposed System
7. Scope of the Project
8. Literature and Technology Background
9. System Architecture
10. Detailed Architecture Explanation
11. Technology Stack
12. On-Chain and Off-Chain Data Design
13. Smart Contract Architecture
14. Smart Contract Interaction
15. Database Design
16. API Architecture
17. Frontend Architecture
18. Backend Architecture
19. Authentication and Authorization
20. Managed Institutional Signing
21. Credential Issuance Workflow
22. Credential Verification Workflow
23. Blockchain Transaction Workflow
24. Hashing and Cryptographic Workflow
25. Credential Lifecycle State Machine
26. Security Analysis
27. Access Control Analysis
28. Unified Audit Trail
29. Analytics Engine
30. Testing Strategy and Results
31. UI/UX Description
32. Limitations
33. Implementation Reality & Important Limitations
34. Future Enhancements
35. Conclusion

36. References
37. Viva Voce Questions and Answers

---

## 1. Introduction

Academic and professional credentials have increasingly moved from paper-based records to digital documents. This transition makes certificates easier to distribute and archive, but it also introduces a basic trust problem. A PDF that visually resembles an authentic university certificate may be modified with commonly available software. A verifier looking at the document cannot reliably determine whether the student's name, degree title, grade information, dates, or other contents have been changed.

Traditional verification portals solve part of this problem by checking a certificate against a centralized institutional database. Such a design is easy to operate but places considerable trust in the institution's application server and database. If the database is modified, compromised, incorrectly administered, or lost, the verification result can also be affected.

CredChain was developed around a different trust model. It does not attempt to place the full credential document on a blockchain. Instead, the blockchain is used as a trust layer that records the minimum cryptographic and lifecycle information needed to determine whether a document corresponds to an authorized credential.

The central design principle of the project is:

«We are not putting certificates on a blockchain. We are building a trusted lifecycle for digital credentials, with blockchain serving as the trust layer.»

The original PDF remains outside persistent storage. During issuance or verification, the PDF is processed as an in-memory buffer and hashed using SHA-256. The resulting digest becomes the cryptographic representation of the document.

Personally identifiable information such as the student's full name is maintained off-chain. The blockchain contains the certificate ID, document digest, issuer wallet, institution binding, lifecycle state, timestamps, expiration information, and version number.

This hybrid approach allows the system to use blockchain where immutability and cryptographic authorization are valuable while avoiding the privacy and cost implications of storing complete documents or personal data on-chain.

The implemented system consists of Solidity smart contracts, a Node.js/Express backend, a SQLite data layer, an administrative React frontend, and lightweight public and issuer web interfaces.

---

## 2. Problem Statement

CredChain addresses five specific problems identified in conventional digital credential management.

### 2.1 Document Forgery and Modification

Digital credentials are normally distributed as PDF files. These files can be edited using standard desktop software. Visual inspection alone therefore cannot establish whether the contents of a PDF remain identical to those originally issued.

CredChain addresses this by computing a SHA-256 digest of the PDF and anchoring the digest to the credential's blockchain record.

### 2.2 Centralized Database Vulnerabilities

A conventional verification portal commonly treats a central database as its authoritative source. Such systems may be affected by application vulnerabilities, database compromise, privileged administrator misuse, accidental modification, or data loss.

CredChain separates application metadata from the authoritative credential trust state. SQLite is used for metadata and analytics, while cryptographically significant credential state is read from the blockchain.

### 2.3 Cross-Institutional Insecurity

A multi-institution credential platform requires stronger isolation than merely storing an institution name beside a certificate. One institution must not be able to change another institution's credentials.

CredChain therefore stores an "institutionId" directly inside each on-chain credential record and verifies this binding when revocation and versioning operations are attempted.

### 2.4 Historical Version Ambiguity

A credential may legitimately be reissued or corrected. A conventional system that overwrites the existing record can remove evidence of earlier versions.

CredChain maintains a monotonically increasing version number and stores blockchain snapshots through the "certificateVersions" mapping.

### 2.5 Verification Friction

A public verifier should not need cryptocurrency, browser wallet software, private keys, or knowledge of blockchain interaction.

CredChain exposes credential verification as a normal public web workflow. Verification is implemented using read-only blockchain calls and therefore does not require a blockchain transaction or gas payment.

---

## 3. Objectives

The primary objective of CredChain is to develop a credential management system in which blockchain acts as a cryptographic trust and lifecycle layer rather than as document storage.

The implementation specifically aims to:

1. Detect modification of issued PDF credentials using SHA-256 document hashing.
2. Restrict blockchain write operations to institution-authorized wallets.
3. Bind credentials cryptographically to their issuing institution.
4. Maintain credential lifecycle states including active, revoked, expired, and versioned records.
5. Preserve historical versions of blockchain credential data.
6. Keep personally identifiable student information outside the blockchain.
7. Allow public verification without requiring MetaMask or transaction fees.
8. Support institutional transaction signing through a managed backend workflow.
9. Maintain a synchronized application-side cache of blockchain events.
10. Record verification activity separately from blockchain transactions.
11. Provide administrative analytics derived from both blockchain state and application telemetry.
12. Demonstrate the architecture using a local Hardhat EVM environment.

Production public-chain deployment is not an objective completed by the current implementation.

---

## 4. Existing System Analysis

Conventional credential verification systems frequently use an institution-controlled web server and relational database. A credential identifier is entered into the portal and the application searches its database for the corresponding record.

This model has several practical strengths. It is easy to build, inexpensive to operate, and familiar to institutional IT teams. However, the verifier is ultimately trusting the same infrastructure that stores and returns the record.

If database values can be changed without an independently verifiable history, the system cannot provide strong evidence that a record is unchanged from its original state.

Another weakness appears when independent institutions share one platform. Application-layer access rules can provide separation, but any defect in authorization logic may potentially expose records belonging to another institution.

A fully decentralized alternative introduces different problems. Requiring employers, students, or external verification agencies to use wallets and blockchain transactions creates unnecessary friction for an operation that is fundamentally a read.

CredChain therefore avoids both extremes. The application still uses conventional web technologies for usability, while blockchain is limited to credential data that benefits directly from immutability and cryptographic authorization.

---

5. Proposed System

CredChain follows a hybrid on-chain/off-chain design.

The blockchain stores the current cryptographic representation and lifecycle state of each credential. SQLite stores student and application metadata. Uploaded PDFs are processed only in volatile application memory.

The on-chain credential record includes:

- certificate identifier;
- SHA-256 document digest;
- issuing wallet address;
- institution identifier;
- issuance timestamp;
- expiration timestamp;
- active or revoked status;
- monotonically increasing version number;
- historical version snapshots; and
- an existence sentinel.

The off-chain application stores:

- student full name;
- course or program title;
- issue date used by the application;
- cached institution and status information;
- verification logs; and
- synchronized blockchain event information.

Raw PDFs are not intentionally persisted by the backend. Private keys are not stored in SQLite. QR codes contain a verification URL rather than personal data or document content.

When a credential is verified, the submitted document is hashed again. The application sends its certificate ID and calculated digest to a read-only smart-contract function. The contract determines the result from the authoritative blockchain state.

---

## 6. Advantages of the Proposed System

CredChain separates the concepts of document storage, metadata storage, and trust.

The first advantage is deterministic tamper detection. The verifier does not depend on visual comparison of PDFs. A file whose digest differs from the active on-chain digest is classified as "TAMPERED".

The second advantage is institutional isolation. Credential lifecycle operations contain an explicit institution binding in addition to issuer authorization.

A third advantage is historical visibility. Creating a replacement credential increments a version counter and records a new version snapshot instead of silently replacing all evidence of the previous blockchain state.

The fourth advantage is public usability. Verification is a blockchain read rather than a transaction. The verifier does not need ETH or a wallet.

The fifth advantage is privacy reduction at the trust layer. Student names and course information are not placed on-chain.

The design also reduces the authority of the SQLite database. Modifying a database row cannot independently create a matching on-chain document digest for a forged PDF.

These properties do not remove the need for secure servers, secure private-key custody, correct input data, backups, or operational controls. They narrow the specific responsibility assigned to the blockchain.

---

## 7. Scope of the Project

The implemented scope covers institution registration, issuer authorization, credential issuance, public verification, revocation, version creation, expiration handling, blockchain event synchronization, verification telemetry, analytics, audit retrieval, PDF generation, and QR-based verification navigation.

The active blockchain environment is a local Hardhat node using Chain ID 31337.

Ethereum Sepolia is configured as a network option, but no deployed Sepolia system is documented.

The current implementation does not include decentralized PDF storage, W3C Verifiable Credential issuance, Decentralized Identifiers, zero-knowledge proofs, Layer-2 deployment, cloud KMS integration, multi-user institutional authentication, or public mainnet/testnet operation.

---

## 8. Literature and Technology Background

### 8.1 Ethereum and EVM Smart Contracts

CredChain uses an Ethereum-compatible execution environment as its trust layer. Solidity smart contracts encode institutional authorization and credential lifecycle rules.

Ethereum is relevant to CredChain because contract state cannot be altered by direct database administration. Credential state changes must occur through defined smart-contract functions and satisfy their authorization conditions.

The current prototype uses Hardhat rather than a public Ethereum network.

### 8.2 Solidity

The three core contracts are written for Solidity "^0.8.20".

Solidity provides the mappings, structures, modifiers, custom errors, events, and contract-to-contract interactions required by CredChain's trust layer.

### 8.3 SHA-256

SHA-256 is used to create the document fingerprint.

The PDF itself is not used as on-chain data. Instead, all bytes of the PDF are processed by SHA-256, producing a 256-bit digest represented by the application as a "0x"-prefixed hexadecimal string.

Any modification that changes the file bytes results in a different digest except for the theoretical possibility of a cryptographic collision. For practical credential integrity checking, a mismatch allows CredChain to classify the uploaded document as "TAMPERED".

### 8.4 JSON Web Tokens

The backend uses JSON Web Tokens for its application authentication layer.

After the configured issuer credentials are validated, a signed JWT containing the username, institution identifier, and "issuer" role is returned. Protected routes verify the Bearer token before continuing.

This JWT layer controls access to backend write endpoints. It is distinct from blockchain authorization, which independently evaluates the signer wallet.

### 8.5 SQLite

SQLite stores data that does not require blockchain immutability, including personal information, verification telemetry, and the application-side event cache.

It was suitable for a local academic prototype because it requires no separate database server and integrates directly with Node.js.

Its use should not be interpreted as a production scalability decision.

8.6 Ethers.js

Ethers.js v6 provides JSON-RPC connectivity, wallet signing, contract bindings, event access, transaction submission, and blockchain read operations.

The Member 2 backend uses it for managed signing, while the Member 1 React interface also uses Ethers.js for direct read operations.

---

9. System Architecture

CredChain contains four main operational layers: user interfaces, backend application services, off-chain persistence, and the blockchain trust layer.

```
graph TD
    U1[Institution Administrator]
    U2[Public Verifier]

    R[Member 1 React Platform<br/>Port 5173]
    V1[Member 2 Issuer Portal<br/>Vanilla JS]
    V2[Member 2 Public Verification Portal<br/>Vanilla JS]

    API[Express.js REST API<br/>Port 3000]
    AUTH[JWT Authentication]
    CERT[Certificate Controller]
    ANA[Analytics Controller]
    AUDIT[Audit Controller]
    HASH[SHA-256 Hash Service]
    SIGN[Managed Institutional Signing]
    SYNC[Event Synchronizer]

    DB[(SQLite<br/>Metadata + Telemetry + Event Cache)]

    FACADE[DigitalCredential.sol<br/>Facade]
    CR[CertificateRegistry.sol]
    IR[InstitutionRegistry.sol]

    EVM[(Hardhat EVM<br/>Chain ID 31337)]
```

```
U1 --> R
U1 --> V1
U2 --> V2
U2 --> R

V1 --> API
V2 --> API
R --> API
R -->|Direct read-only JSON-RPC| FACADE

API --> AUTH
API --> CERT
API --> ANA
API --> AUDIT

CERT --> HASH
CERT --> SIGN
CERT --> DB

ANA --> DB
ANA --> FACADE
AUDIT --> DB

SIGN --> FACADE
FACADE --> CR
FACADE --> IR
CR --> IR

FACADE --> EVM
CR --> EVM
IR --> EVM

EVM --> SYNC
SYNC --> DB
```

The blockchain remains authoritative for credential trust state. SQLite improves usability, analytics, and application indexing without replacing the blockchain as the cryptographic source of truth.

---

10. Detailed Architecture Explanation

10.1 Member 1 Subsystem

Member 1 owns the blockchain core and trust layer.

The smart-contract portion contains:

- "InstitutionRegistry.sol"
- "CertificateRegistry.sol"
- "DigitalCredential.sol"

The same subsystem also contains deployment scripts, demonstration setup logic, compiled artifacts, and eight Hardhat test files.

Member 1 additionally owns the React/Vite administrative frontend. This interface exposes dashboards, credential management, institution information, issuer information, verification, analytics, blockchain activity, settings, and credential detail views.

10.2 Member 2 Subsystem

Member 2 owns the blockchain application and verification layer.

The backend contains:

- Express bootstrap;
- blockchain configuration;
- SQLite initialization;
- credential controllers;
- authentication;
- analytics;
- audit functionality;
- SHA-256 hashing;
- JWT middleware; and
- historical plus real-time blockchain event synchronization.

Member 2 also provides two lightweight web portals served directly by Express:

- an authenticated issuer portal; and
- a zero-authentication public verification portal.

10.3 Project Organization

The project is divided into:

```
Certificate-Verification-System/
├── README.md
├── CredChain_Report.docx
├── member-1-blockchain-core/
│   ├── contracts/
│   ├── scripts/
│   ├── test/
│   ├── artifacts/
│   └── frontend/
└── member-2-blockchain-application/
```

```
├── server.js
├── database.sqlite
├── src/
├── public/
└── test_clean_canonical_audit.js
```

This organization reflects the two-member technical ownership model rather than separating the project only by programming language.

---

11. Technology Stack

Layer| Technology| Version| Role
Smart Contracts| Solidity| ^0.8.20| On-chain business logic
EVM Development| Hardhat| ^2.29.1| Local node, compiler, test runner
Contract Security| OpenZeppelin Contracts| ^5.6.1| Ownable for InstitutionRegistry
Blockchain Client — Backend| Ethers.js| ^6.11.1| JSON-RPC, contract bindings, signing
Blockchain Client — Frontend| Ethers.js| ^6.17.0| Direct JSON-RPC reads
Backend Runtime| Node.js ESM| v18+| Server runtime
Web Framework| Express.js| ^4.18.2| REST API server
File Processing| Multer| ^1.4.5-lts.1| In-memory PDF processing
Off-Chain Database| SQLite3| ^5.1.7| Metadata, events, telemetry
Authentication| jsonwebtoken| ^9.0.2| Bearer token authentication
QR Code Engine| qrcode| ^1.5.3 / ^1.5.4| Verification URL generation
Administrative UI| React| ^19.2.8| Member 1 platform
Build Tool| Vite| ^8.2.2| React build tooling
Routing| react-router-dom| ^7.18.3| SPA routing
PDF Engine| jsPDF| ^4.2.1| Client-side A4 certificate generation
Icons| Lucide React| ^1.41.0| UI iconography
Date Utilities| date-fns| ^4.4.0| Date formatting
Backend Cryptography| Node.js crypto| Native| SHA-256 hashing
Browser Cryptography| Web Crypto API| Native| SHA-256 hashing
Public UI| HTML5/CSS3/JavaScript| —| Lightweight portals
Environment Configuration| dotenv| ^16.4.5| ".env" loading
CORS| cors| ^2.8.5| Express CORS middleware
Testing| Mocha/Chai via hardhat-toolbox| ^6.1.2| Smart-contract tests

The primary active network is the Hardhat localhost network at "http://127.0.0.1:8545", Chain ID "31337".

Ethereum Sepolia, Chain ID "11155111", is configured but is not documented as a live deployment.

---

12. On-Chain and Off-Chain Data Design

Data| Storage| Authoritative Source| Reason
Certificate ID| Blockchain + SQLite| Blockchain| Identifier for cryptographic lookup
SHA-256 Hash| Blockchain| Blockchain| Immutable document anchor
Issuer Wallet| Blockchain| Blockchain| Identifies transaction signer
Institution ID| Blockchain + SQLite| Blockchain| Institutional security boundary
Status| Blockchain + SQLite| Blockchain| Lifecycle state
Version| Blockchain| Blockchain| Monotonic revision sequence
Version Snapshots| Blockchain| Blockchain| Historical lifecycle record
Expiration Timestamp| Blockchain| Blockchain| Compared with block time
Student Full Name| SQLite| SQLite| Avoids putting PII on-chain
Course Title| SQLite| SQLite| Application display metadata
Raw PDF| RAM only| Not persisted| Privacy and storage reduction
Verification Telemetry| SQLite| SQLite| Analytics and auditing
Blockchain Event Cache| SQLite| Blockchain| Faster querying and UI access

The privacy design reduces the amount of personal data exposed on the blockchain. However, storing PII off-chain should be described as a privacy-oriented architecture rather than proof of full GDPR or FERPA compliance. Regulatory compliance also depends on operational, legal, retention, access-control, and governance practices that are outside the inspected implementation.

---

13. Smart Contract Architecture

13.1 InstitutionRegistry.sol

"InstitutionRegistry.sol" manages institution identity and issuer authorization.

It inherits OpenZeppelin "Ownable", making the deploying owner the platform-level administrator.

The principal mappings are:

```
mapping(string => Institution) private institutions;
string[] private institutionIds;
mapping(string => mapping(address => bool)) private authorizedIssuers;
```

An institution contains its ID, name, authority wallet, activity status, and existence flag.

Only the contract owner can register and deactivate institutions.

Issuer management is intentionally delegated downward. "authorizeIssuer()" and "revokeIssuer()" may only be invoked by the wallet registered as the institution's authority wallet.

The function:

isAuthorizedIssuer(institutionId, issuer)

returns false when the institution itself is inactive. Institution deactivation therefore disables issuer authorization without requiring every issuer entry to be individually removed.

Events include institution registration and deactivation as well as issuer authorization and revocation.

The contract uses custom errors such as "InstitutionAlreadyExists", "InstitutionInactive", and "UnauthorizedCaller".

13.2 CertificateRegistry.sol

"CertificateRegistry.sol" implements the credential lifecycle.

Its central structure is:

```
struct Certificate {
    string certificateId;
    string certificateHash;
    address issuer;
    uint256 issueTimestamp;
    uint256 expiryTimestamp;
    CertificateStatus status;
    uint256 version;
    bool exists;
    string institutionId;
}
```

The current record is stored in:

```
mapping(string => Certificate) private certificates;
```

Historical records are stored in:

```
mapping(string => mapping(uint256 => Certificate))
    private certificateVersions;
```

Issuance

"issueCertificate()" confirms that the supplied caller is authorized for the supplied institution.

It rejects empty certificate IDs or hashes and rejects duplicate IDs.

A successful issuance creates version 1 with status "ACTIVE", sets "issueTimestamp" using "block.timestamp", stores the first version snapshot, and emits "CertificateIssued".

Verification

"verifyCertificate()" is a view function and therefore does not create a transaction.

The evaluation order is exact:

1. "NOT_FOUND"
2. "TAMPERED"
3. "REVOKED"
4. "EXPIRED"
5. "VALID"

This order has visible consequences.

For example, if a credential is revoked but the submitted PDF has also been modified, the result is "TAMPERED", because the hash check occurs before revocation status is evaluated.

Likewise, a modified expired credential is reported as "TAMPERED", whereas an unmodified credential that is both revoked and expired is reported as "REVOKED".

Revocation

"revokeCertificate()" first confirms that the certificate exists.

It then checks whether the certificate's stored institution ID matches the institution ID supplied for the operation.

After the institution boundary is validated, the caller must satisfy the implemented authorization logic: original issuer, currently authorized issuer, or institution authority wallet.

A second revocation attempt is rejected using "CertificateAlreadyRevoked".

Revocation is permanent in the documented implementation.

Versioning

"createNewVersion()" validates the certificate, institution binding, and caller.

The active credential hash is replaced with the new digest, the version number increments, and "issueTimestamp" is refreshed to the current block timestamp. A snapshot of the new version is then stored in "certificateVersions".

Consequently, the standard verification function validates only the currently active document hash. An earlier PDF does not remain "VALID" merely because its historical snapshot exists. Once version 2 becomes active, the version 1 file is classified as "TAMPERED" by the normal current-state verification function.

Historical data can instead be inspected using "getCertificateVersion()".

The documented implementation also propagates the current revoked state when historical versions are retrieved.

13.3 DigitalCredential.sol

"DigitalCredential.sol" implements a facade.

Instead of requiring application components to coordinate direct calls to multiple internal contracts, it provides a single external interface.

For writes, it forwards "msg.sender" to "CertificateRegistry" as an explicit caller argument.

For example:

```
issueCertificate(...)
   =>
certificateRegistry.issueCertificate(msg.sender, ...)
```

This preserves the original signer identity even though the registry sees the facade as the immediate contract caller.

Read operations are also forwarded for institutions, issuer authorization, certificates, versions, counts, and verification.

13.4 onlyFacade Modifier

"CertificateRegistry" contains:

```
modifier onlyFacade() {
   if (
      msg.sender != facadeAddress &&
      facadeAddress != address(0)
   ) revert OnlyFacadeAllowed();
   _;
}
```

Once "facadeAddress" is configured, lifecycle writes must pass through the expected facade address.

One implementation detail is important from a security perspective: while "facadeAddress" remains the zero address, the condition permits calls because the second part of the rejection expression is false. Correct initialization of the facade address is therefore security-sensitive. The supplied project description does not document a separate owner-only bootstrap restriction for "setFacadeAddress()" beyond the shown modifier.

---

14. Smart Contract Interaction

```
sequenceDiagram
    actor Issuer
    participant Facade as DigitalCredential.sol
    participant Cert as CertificateRegistry.sol
    participant Inst as InstitutionRegistry.sol
    participant EVM as EVM State

    Issuer->>Facade: issueCertificate(instId, certId, hash, expiry)
    Facade->>Cert: issueCertificate(msg.sender, instId, certId, hash, expiry)

    Cert->>Inst: isAuthorizedIssuer(instId, caller)
    Inst-->>Cert: true / false

    alt Unauthorized
        Cert-->>Facade: revert UnauthorizedIssuer
        Facade-->>Issuer: transaction reverted
    else Authorized
        Cert->>EVM: Store Certificate v1
        Cert->>EVM: Store certificateVersions[id][1]
        Cert->>EVM: Emit CertificateIssued
        Cert-->>Facade: success
        Facade-->>Issuer: confirmed transaction
    end
```

The facade centralizes access while the registry retains actual lifecycle rules.

---

15. Database Design

CredChain's SQLite database contains three tables.

```
erDiagram
    CERTIFICATES {
        TEXT id PK
        TEXT studentName
        TEXT courseName
        TEXT issueDate
        TEXT institutionId
        TEXT status
    }

    VERIFICATION_LOGS {
        INTEGER id PK
        TEXT certificateId
        TEXT timestamp
```

```
    TEXT status
    TEXT ipAddress
    TEXT userAgent
  }

  BLOCKCHAIN_EVENTS {
    INTEGER id PK
    TEXT eventType
    TEXT certificateId
    TEXT institutionId
    TEXT issuer
    TEXT timestamp
    INTEGER blockNumber
    TEXT transactionHash
    INTEGER logIndex
    INTEGER version
  }

  CERTIFICATES ||--o{ VERIFICATION_LOGS : "logical certificateId association"
  CERTIFICATES ||--o{ BLOCKCHAIN_EVENTS : "logical certificateId association"
```

These relationships are logical associations based on "certificateId". The supplied SQL schema does not define foreign-key constraints between the tables.

This distinction matters because SQLite does not automatically guarantee referential consistency between these records through the shown schema.

15.1 certificates

The "certificates" table stores application-facing information:

```
CREATE TABLE IF NOT EXISTS certificates (
  id TEXT PRIMARY KEY,
  studentName TEXT,
  courseName TEXT,
  issueDate TEXT,
  institutionId TEXT,
  status TEXT
);
```

It contains personal and display information that is intentionally kept off-chain.

The status is duplicated for application convenience, but the blockchain remains authoritative.

15.2 verification_logs

The verification log records:

- certificate identifier;
- timestamp;
- verification result;
- IP address; and
- user agent.

These are application operations and are not blockchain transactions.

15.3 blockchain_events

The event cache stores blockchain event information including block number, transaction hash, log index, institution, issuer, event type, and version.

A uniqueness constraint on:

(transactionHash, logIndex)

allows event synchronization to be idempotent.

Four indexes are created for event type, certificate, institution, and timestamp.

---

16. API Architecture

Authentication

Method| Endpoint| Authentication| Purpose
POST| "/api/auth/login"| No| Validate credentials and return JWT

Certificate Lifecycle

Method| Endpoint| Authentication| Purpose
POST| "/api/certificates/issue"| JWT| Issue a credential
POST| "/api/certificates/verify"| No| Verify uploaded PDF
POST| "/api/certificates/revoke"| JWT| Revoke a credential
POST| "/api/certificates/version"| JWT| Create a new credential version
GET| "/api/certificates"| No| Retrieve cached credentials
GET| "/api/certificates/:id"| No| Retrieve one cached record
GET| "/api/certificates/:id/audit"| JWT| Retrieve verification logs

Analytics

Method| Endpoint| Authentication
GET| "/api/analytics/summary"| No
GET| "/api/analytics/issuance-trends"| No
GET| "/api/analytics/verification-trends"| No

GET| "/api/analytics/verification-results"| No
GET| "/api/analytics/institutions"| No
GET| "/api/analytics/recent-activity"| No

Audit

Method| Endpoint| Authentication| Parameters
GET| "/api/audit/events"| JWT| "page", "limit", "eventType", "source", "search"

The APIs separate public verification from credential-changing operations. Verification requires no JWT because it does not change blockchain state.

---

17. Frontend Architecture

CredChain contains two frontend approaches.

17.1 React Administration Platform

Member 1's frontend uses React 19 with Vite.

Its routes are:

/ Public landing page
/verify Public PDF verification
/dashboard Summary metrics
/analytics Trend and institution analytics
/institutions Institution directory
/issuers Issuer management
/credentials Credential catalog and issuance
/credentials/:id Lifecycle and version details
/activity Blockchain activity
/verification Administrative verification interface
/settings RPC/network configuration

The interface uses custom CSS through "tokens.css", "layout.css", and "components.css". Tailwind CSS is not part of the implementation.

A light/dark preference is persisted in "localStorage".

The frontend can detect MetaMask through an injected Ethereum provider and listens to account and chain changes. This functionality is optional for direct blockchain interaction. Public verification and institutional issuance are not designed to require a MetaMask confirmation workflow.

The "blockchain.js" service uses Ethers.js v6 and direct JSON-RPC for blockchain reads.

17.2 PDF Generation

"pdfGenerator.js" creates an A4 landscape credential using jsPDF.

The generated document includes institutional information, student information, course information, certificate ID, institution ID, issue date, optional expiry information, and a QR code.

The QR value is:

{origin}/verify?id={certificateId}

It therefore contains navigation information rather than the student's personal data.

The browser computes the generated PDF digest using:

crypto.subtle.digest('SHA-256', pdfArrayBuffer)

17.3 Member 2 Public Portal

"verify.html" provides the lowest-friction verification experience.

A user can:

- upload a PDF;
- optionally enter a certificate ID;
- arrive through a QR-generated "?id=" URL; and
- receive one of the supported verification states.

No login is required.

17.4 Member 2 Issuer Portal

"index.html" provides JWT-authenticated issuance and revocation workflows.

After successful operations, it can display the blockchain transaction hash, block number, SHA-256 digest, and QR code.

---

18. Backend Architecture

The Member 2 backend uses an Express modular structure.

The application startup sequence is:

initializeDatabase()
↓

connectBlockchain()
↓
ensureInstitutionalAuthorizations()
↓
startEventSynchronizer()
↓
app.listen(3000)

"server.js" configures:

cors()
express.json()
express.urlencoded()
express.static('public')

and mounts the authentication, certificate, analytics, and audit routes.

18.1 blockchain.js

This module creates the JSON-RPC provider and contract objects.

It also supplies institutional wallets using "getInstitutionSigner()".

Local development keys are mapped to institution identifiers through "DEV_INSTITUTION_KEYS" or environment variables.

These keys correspond to Hardhat accounts and are explicitly development-only.

A production system would require a different custody architecture. Cloud KMS and MPC are documented future/production directions but are not implemented.

18.2 certificateController.js

The controller implements the major lifecycle operations.

For issuance it validates input, computes the digest when a file is supplied, performs an on-chain authorization pre-flight, obtains the correct institution signer, manages the nonce, submits the facade call, waits for confirmation, records metadata, and creates a verification QR code.

For verification it calculates a new PDF digest and calls the blockchain's read-only verification path.

For revocation and versioning it obtains the institution signer and invokes the corresponding facade functions.

18.3 Certificate ID Extraction

When the verifier does not provide a certificate ID explicitly, the backend attempts to extract one from the PDF buffer.

It checks:

1. verification URL patterns;
2. textual "Certificate ID:" patterns; and
3. known certificate prefixes.

This is a heuristic mechanism. It should not be treated as a general PDF semantic parser.

---

19. Authentication and Authorization

CredChain has two separate authorization layers.

19.1 Application Authentication

The Express API uses JWT.

Login credentials are compared against environment-configured values:

ISSUER_USERNAME
ISSUER_PASSWORD

No password hashing library is used.

A successful login creates a token containing:

```
{
   username,
   institutionId,
   role: "issuer"
}
```

with a 24-hour expiration.

Protected API routes expect:

Authorization: Bearer <token>

A "TokenExpiredError" results in HTTP 403.

Token expiration limits the useful lifetime of a stolen token, but it is not a complete replay-prevention mechanism. The documented implementation does not describe token revocation, one-time token identifiers, refresh-token rotation, or a replay blacklist.

19.2 Blockchain Authorization

Blockchain authorization is separate and stronger with respect to lifecycle state.

A valid JWT alone cannot make an arbitrary wallet an institution's authorized blockchain issuer.

The blockchain verifies the signer against "InstitutionRegistry".

This layered design means:

Application authentication
+
Institution-specific blockchain signer
+
On-chain authorization check

must align for a normal protected write operation to succeed.

---

20. Managed Institutional Signing

One of the central usability decisions in CredChain is managed institutional transaction signing.

In many Web3 applications, a user presses an action button and receives a MetaMask confirmation dialog.

CredChain's institutional workflow instead asks the backend to select the configured signer for the institution.

The backend performs:

Institution ID
↓
getInstitutionSigner()
↓
ethers.Wallet
↓
DigitalCredential contract connected to wallet
↓
Signed transaction
↓
Hardhat EVM

This removes the need for the operational user to understand gas, networks, transaction nonces, or wallet confirmations.

The design does not remove private-key risk. It moves custody responsibility from the user's browser to backend infrastructure.

In the current project, the signing setup is explicitly development-grade because local Hardhat keys are available in "DEV_INSTITUTION_KEYS".

---

21. Credential Issuance Workflow

The complete implemented issuance path is:

1. The administrator opens the issuer interface.
2. The administrator logs in and receives a JWT.
3. Credential data and a PDF are provided.
4. The client sends a multipart request to "/api/certificates/issue".
5. JWT middleware validates the Bearer token.
6. The backend computes SHA-256 over the PDF buffer.
7. The intended signer is checked against the institution's on-chain authorization.
8. The backend loads the institutional signer.
9. A live pending nonce is obtained.
10. A transaction is sent to "DigitalCredential.issueCertificate()".
11. The facade forwards the original signer to "CertificateRegistry".
12. "CertificateRegistry" verifies authorization with "InstitutionRegistry".
13. Version 1 of the credential is stored.
14. "CertificateIssued" is emitted.
15. The backend waits for transaction confirmation.
16. Event information is persisted in the SQLite event cache.
17. Student/course metadata is inserted in SQLite.
18. A public verification URL and QR code are generated.
19. The response returns the credential identifier, digest, transaction information, issuer, institution, QR code, and verification URL.

```
sequenceDiagram
    actor Admin
    participant UI as Issuer UI
    participant API as Express API
    participant Auth as JWT Middleware
    participant Hash as SHA-256 Service
    participant BC as blockchain.js
    participant Facade as DigitalCredential
    participant Cert as CertificateRegistry
    participant Inst as InstitutionRegistry
    participant DB as SQLite

    Admin->>UI: Login + credential data + PDF
    UI->>API: POST /api/certificates/issue
```

```
    API->>Auth: Validate Bearer token
    Auth-->>API: Authorized

    API->>Hash: computeSHA256(PDF buffer)
    Hash-->>API: 0x-prefixed digest

    API->>BC: verifyInstitutionSignerOnChain()
    BC->>Inst: isAuthorizedIssuer()
    Inst-->>BC: authorization result

    API->>BC: get institution signer + live nonce
    BC->>Facade: issueCertificate(...)
    Facade->>Cert: issueCertificate(msg.sender,...)
    Cert->>Inst: isAuthorizedIssuer()
    Inst-->>Cert: true
    Cert-->>BC: CertificateIssued + transaction receipt

    API->>DB: Store event cache
    API->>DB: Store off-chain metadata
    API-->>UI: certId, hash, txHash, block, QR, URL
```

---

22. Credential Verification Workflow

Verification intentionally avoids blockchain transactions.

The workflow is:

1. The verifier opens the public verification page.
2. A PDF is selected.
3. A certificate ID can be entered manually or supplied through the QR URL.
4. The browser sends the PDF to "/api/certificates/verify".
5. The backend calculates SHA-256.
6. If the ID was omitted, the application attempts heuristic extraction.
7. "verifyCertificate(certId, hash)" is called through JSON-RPC.
8. The contract evaluates existence, digest, revocation, expiration, and validity.
9. "getCertificate()" is read to obtain the current version.
10. Verification telemetry is inserted into SQLite.
11. The result is returned to the verifier.

```
sequenceDiagram
    actor Verifier
    participant UI as verify.html / React
    participant API as Express API
    participant Hash as SHA-256 Service
    participant Facade as DigitalCredential
    participant Cert as CertificateRegistry
```

```
    participant DB as SQLite

    Verifier->>UI: Upload PDF
    UI->>API: POST /api/certificates/verify
    API->>Hash: Hash PDF buffer
    Hash-->>API: SHA-256 digest

    opt Certificate ID missing
        API->>API: Attempt regex/prefix extraction
    end

    API->>Facade: verifyCertificate(certId, hash)
    Facade->>Cert: verifyCertificate(certId, hash)
    Cert-->>Facade: status
    Facade-->>API: status

    API->>Facade: getCertificate(certId)
    Facade-->>API: current version

    API->>DB: Insert verification telemetry
    API-->>UI: certificateId, status, version
    UI-->>Verifier: Display result
```

Because this path consists of view operations, the public verifier does not need a wallet and does not pay gas.

---

23. Blockchain Transaction Workflow

Credential-changing operations follow a transaction lifecycle.

The application first identifies the institution involved. It obtains the corresponding configured Ethers.js wallet and verifies that its address is authorized on-chain.

A pending nonce is fetched to reduce the risk of nonce collisions between backend submissions.

The backend then connects the "DigitalCredential" contract to the institutional signer and invokes the appropriate function.

Ethers.js signs the transaction using the institution's private key.

The transaction is submitted to the local JSON-RPC endpoint.

The backend waits for:

tx.wait()

before treating the blockchain operation as confirmed.

The receipt provides transaction-level information such as the block number and transaction hash.

This transaction workflow applies to state-changing operations such as issuance, revocation, and version creation.

Verification does not follow this flow because it is a read-only contract operation.

---

24. Hashing and Cryptographic Workflow

The backend implements hashing as:

```
import crypto from 'crypto';

export function computeSHA256(buffer) {
    const hash = crypto
        .createHash('sha256')
        .update(buffer)
        .digest('hex');

    return `0x${hash}`;
}
```

The browser implementation uses:

```
crypto.subtle.digest(
    'SHA-256',
    pdfArrayBuffer
)
```

and converts the result to the same hexadecimal representation.

The blockchain stores the resulting SHA-256 value as a Solidity "string".

To compare two Solidity strings, "CertificateRegistry" hashes their byte representations:

```
keccak256(bytes(cert.certificateHash))
    !=
keccak256(bytes(_certificateHash))
```

The purpose of this second hash is not to replace SHA-256.

The cryptographic chain is:

PDF bytes
  ↓
SHA-256
  ↓
0x-prefixed digest string
  ↓
Stored in Certificate struct
  ↓
keccak256(bytes(string)) used for EVM string equality comparison

The document fingerprint remains SHA-256.

"keccak256" is being used inside Solidity as a convenient deterministic comparison of dynamically sized byte sequences.

---

25. Credential Lifecycle State Machine

```
stateDiagram-v2
    [*] --> NEW

    NEW --> ACTIVE_V1: issueCertificate()

    ACTIVE_V1 --> ACTIVE_V1: verify original PDF / VALID
    ACTIVE_V1 --> TAMPERED: verify modified PDF
    ACTIVE_V1 --> EXPIRED: block.timestamp > expiryTimestamp
    ACTIVE_V1 --> ACTIVE_V2: createNewVersion()

    ACTIVE_V2 --> ACTIVE_V2: verify v2 PDF / VALID
    ACTIVE_V2 --> TAMPERED: verify v1 or modified PDF

    ACTIVE_V1 --> REVOKED: revokeCertificate()
    ACTIVE_V2 --> REVOKED: revokeCertificate()

    REVOKED --> REVOKED: irreversible

    NEW --> NOT_FOUND: unknown certificate ID

    note right of REVOKED
        With correct current hash:
        result = REVOKED.
        With incorrect hash:
        result = TAMPERED first,
        due to evaluation order.
    end note
```

A separate historical version mapping preserves version snapshots, but normal verification always compares against the current active hash.

---

26. Security Analysis

Threat| Implemented Defense| Component
PDF modification| SHA-256 mismatch produces "TAMPERED"| CertificateRegistry
Unauthorized issuance| Institution/issuer authorization checked on-chain| InstitutionRegistry + CertificateRegistry
Cross-institution modification| Stored institution binding compared before lifecycle changes| CertificateRegistry
SQLite credential manipulation| Verification obtains trust state from blockchain| Backend + contracts
Duplicate revocation| "CertificateAlreadyRevoked" custom error| CertificateRegistry
Browser exposure of institutional keys| Managed signing keeps issuer keys out of normal browser workflow| blockchain.js
PII disclosure through QR| QR contains verification URL only| PDF/QR generation
Unauthorized protected API use| JWT Bearer middleware| authMiddleware.js
Long-lived stolen JWT| 24-hour expiration bounds token lifetime| JWT configuration
Direct registry lifecycle writes after facade configuration| "onlyFacade" restriction| CertificateRegistry
Empty required credential data| Validation before blockchain operation| certificateController.js
Hashed indexed string event topics| Resolution through known IDs and "ethers.id()"| eventListener.js

Several qualifications are necessary.

First, the 24-hour JWT expiration limits replay exposure but does not make a stolen token non-replayable during its validity period.

Second, private keys are not normally exposed to the browser, but local development keys are present in the backend's development configuration. The current design is therefore not production-grade private-key custody.

Third, the blockchain protects integrity of values after they are accepted. It cannot establish that an institution entered truthful academic information.

Fourth, because the deployment is a local Hardhat network, blockchain persistence is limited by the lifecycle of that development chain. Resetting the local chain removes its state unless external persistence or redeployment procedures recreate it.

---

27. Access Control Analysis

CredChain implements hierarchical blockchain roles.

```
graph TD
    A[Platform Administrator<br/>Ownable Contract Owner]
    I[Institution Authority Wallet]
    O[Operational Issuer Wallet]
    C[Credential Lifecycle]

    A -->|registerInstitution / deactivateInstitution| I
    I -->|authorizeIssuer / revokeIssuer| O
    O -->|issue / revoke / create version| C
```

At the highest level, the platform owner can register or deactivate institutions.

Institution authority wallets control which operational addresses are recognized as issuers for their institution.

Operational issuer wallets perform normal credential lifecycle activities.

The certificate itself retains its institution binding, which is checked again during relevant lifecycle operations.

This design provides two dimensions of authorization:

Who is the wallet?
+
For which institution is that wallet authorized?

The institution identifier is therefore part of the security boundary rather than merely a descriptive label.

---

28. Unified Audit Trail

CredChain treats blockchain changes and application verification activity as different categories.

28.1 Blockchain State Events

Seven event types are synchronized:

CertificateIssued
CertificateRevoked
CertificateVersionCreated
InstitutionRegistered
InstitutionDeactivated
IssuerAuthorized

IssuerRevoked

The cached fields can include event type, certificate, institution, issuer, timestamp, block number, transaction hash, log index, and version.

These originate from blockchain state changes.

28.2 Application Verification Events

Verification is a read operation.

It therefore has:

- no transaction hash;
- no blockchain block inclusion; and
- no gas cost.

Instead, the application stores:

certificateId
timestamp
status
ipAddress
userAgent

in "verification_logs".

CredChain deliberately avoids presenting these read operations as fabricated blockchain transactions.

28.3 Event Synchronization

"eventListener.js" performs two tasks.

First, "backfillHistoricalEvents()" queries contract events from block zero to the latest block.

Second, live listeners continue receiving new events as they occur.

Indexed Solidity strings require additional handling because indexed dynamic values appear in event topics as hashes rather than their original strings.

The synchronizer attempts to recover the underlying certificate or institution ID using known on-chain IDs, SQLite data, previous events, and "ethers.id()" comparisons.

A fallback raw topic hash is used if resolution is unavailable.

Block timestamps are cached in an in-memory map to reduce repeated block lookups.

---

29. Analytics Engine

CredChain's analytics are hybrid.

Some values are calculated directly from blockchain state, while others come from SQLite telemetry and the synchronized event cache.

"getSummary()" retrieves certificate identifiers and their current on-chain state. It also inspects institution information and issuer authorization.

Issuance trends are created from synchronized "CertificateIssued" events.

Verification trends and result distributions are derived from "verification_logs".

Institution-level credential statistics combine cached application data with on-chain institution validation.

Recent activity returns recent issuance, revocation, and verification information.

This architecture provides a practical distinction between truth and query efficiency.

The blockchain answers questions about authoritative credential state. SQLite is used when the required information is application telemetry or when an indexed cache is more appropriate for UI presentation.

Analytics freshness therefore depends partly on successful event synchronization.

---

30. Testing Strategy and Results

The blockchain core contains eight documented Hardhat test files.

Test File| Tests| Main Coverage
Authorization.test.js| 5| Issuer authorization, issuer revocation, unauthorized management, inactive institutions
CertificateRegistry.test.js| 7| Issuance, duplicates, validation, authorization, VALID/TAMPERED/NOT_FOUND
CrossInstitutionSecurity.test.js| 3| Institution B attacks Institution A and legitimate Institution A operations
Expiration.test.js| 2| Valid before expiry and expired afterward
InstitutionIssuanceE2E.test.js| 2| Unauthorized rejection and authorized end-to-end issuance
InstitutionRegistry.test.js| 8| Registration, duplicates, invalid data, authorization, retrieval, deactivation
Revocation.test.js| 4| Revocation, authority wallet operation, unauthorized attempts, duplicate protection

Versioning.test.js| 2| Version creation/counter and unauthorized version attempts
Total| 33| All documented as passing

The reported execution time is approximately four seconds in the supplied project description.

The test suite covers both successful workflows and rejection conditions.

Of particular importance are the cross-institution tests because they validate one of the project's primary security requirements: an authorized participant in one institution must not automatically gain authority over another institution's credentials.

Member 2 also contains:

test_clean_canonical_audit.js

which is described as a forensic lifecycle and security isolation test.

Detailed assertion counts or execution results for this additional file are not available beyond the supplied description.

---

31. UI/UX Description

CredChain deliberately provides different levels of user-interface complexity for different audiences.

The React interface targets administrators and system operators. It provides dashboards, analytics, credential details, blockchain activity, institution and issuer information, and settings.

The issuer portal offers a simpler workflow for login, issuance, and revocation.

The verification portal is intentionally minimal. A verifier does not need to understand blockchain addresses, RPC endpoints, gas, smart contracts, or wallet networks.

The QR code contributes to this simplification.

Rather than storing sensitive data, it points the verifier to:

/verify?id=<certificateId>

The certificate ID can therefore be pre-filled while the actual PDF still has to be supplied for cryptographic comparison.

A light/dark mode is supported by the React platform and persisted in browser local storage.

---

32. Limitations

The current implementation has several documented limitations.

The most important is the use of a local Hardhat network. The project has not demonstrated persistent operation on a public testnet or mainnet.

Private-key management is also appropriate only for development. Hardhat account keys are available to backend configuration and should be replaced with a production custody mechanism.

Authentication consists of a single configured username and password. There is no per-user account database or per-institution login model.

The configured password is compared as plain text against an environment variable. There is no bcrypt or Argon2 password hashing.

SQLite is appropriate for this prototype but is not designed as the final database for a write-heavy, highly concurrent deployment.

Certificate ID extraction from arbitrary PDF bytes is heuristic.

The application cannot recover a lost PDF because it stores the digest rather than the document.

On a public blockchain, issuance, revocation, and version creation would require gas. An ERC-4337 paymaster or comparable gas sponsorship mechanism has not been implemented.

The project does not currently use IPFS, Arweave, zero-knowledge proofs, or Layer-2 networks.

---

33. Implementation Reality & Important Limitations

This section distinguishes the implemented prototype from possible production extensions.

33.1 Network: Local Only

The active deployment uses a Hardhat localhost node at:

http://127.0.0.1:8545

with Chain ID:

31337

Sepolia configuration exists, but no live Sepolia deployment is documented.

No claim of public-blockchain deployment should therefore be made.

33.2 Authentication Is Simplified

The backend has one environment-configured administrative username and password.

It does not implement a user database, individual institution accounts, password reset, role administration, or multi-user access control at the application level.

33.3 No Password Hashing

Authentication compares the submitted password directly with an environment-configured string.

No bcrypt, Argon2, scrypt, or equivalent password hashing implementation is present.

33.4 Development-Grade Key Management

Institutional private keys are represented through development configuration including "DEV_INSTITUTION_KEYS".

These values are suitable for Hardhat testing only.

Cloud KMS or MPC custody is a recommended production direction, not an implemented feature.

33.5 SQLite Is Not the Production Scalability Layer

SQLite provides a simple embedded persistence model appropriate for demonstration and development.

No claim is made that the current database architecture is suitable for high-concurrency institutional production workloads.

33.6 Analytics Freshness Depends on Synchronization

Analytics combine direct blockchain reads with cached blockchain events and application telemetry.

If the event cache is empty or a fresh blockchain instance has started, some analytics can remain incomplete until historical backfilling has finished.

33.7 PDF Certificate-ID Extraction Is Heuristic

The extraction mechanism scans PDF bytes for known URL patterns, text patterns, and known certificate prefixes.

It is not a full general-purpose PDF semantic extraction system.

Manual ID entry may therefore be needed for documents that do not follow expected CredChain formatting.

33.8 No Decentralized Document Storage

No raw certificate document is uploaded to IPFS or Arweave.

PDFs are processed in application memory and are not deliberately persisted by the issuance/verification implementation described here.

33.9 No Zero-Knowledge Proof System

CredChain does not currently provide private selective disclosure through ZK proofs.

This remains future scope.

33.10 Blockchain Trust Is Deliberately Limited

The blockchain can provide cryptographically controlled state transitions and tamper evidence for the document digest.

It cannot determine whether a university entered correct academic information.

It cannot recover a deleted PDF.

It cannot prevent every compromise of the application server or private-key environment.

It also cannot make an ephemeral Hardhat development chain equivalent to the durability of a deployed public blockchain.

The implementation should therefore be described as a blockchain-backed trust architecture rather than as an absolute security guarantee.

---

34. Future Enhancements

34.1 Decentralized Document Storage

Encrypted credential documents could later be archived through systems such as IPFS or Arweave.

This is not implemented in the current system.

### 34.2 W3C Verifiable Credentials and DIDs

Support for W3C Verifiable Credentials and decentralized identity standards could allow CredChain credentials to interoperate with compatible identity wallets and external verification ecosystems.

### 34.3 Layer-2 Deployment

Future deployment could evaluate Layer-2 networks such as Arbitrum, Optimism, or Polygon to reduce transaction costs compared with Ethereum mainnet operation.

No Layer-2 integration is currently implemented.

### 34.4 Zero-Knowledge Proofs

A future selective-disclosure model could allow a credential holder to prove a property without exposing all credential attributes.

No ZKP implementation exists in the inspected project.

### 34.5 Production Key Custody

Backend-held development keys should be replaced by hardware-backed or managed signing infrastructure such as cloud KMS or MPC custody before production deployment.

---

## 35. Conclusion

CredChain demonstrates a hybrid approach to digital credential verification in which blockchain is used selectively rather than as a general-purpose storage platform.

The project keeps the credential document itself outside the blockchain. Instead, the system records the SHA-256 document fingerprint, credential identity, institution binding, issuer address, lifecycle state, timestamps, and version information on an EVM-compatible smart-contract layer.

The design separates personal student information from cryptographic trust data. This makes it possible to verify document integrity without permanently exposing the student's name or full credential content through blockchain storage.

InstitutionRegistry establishes institutional authority and operational issuer permissions. CertificateRegistry implements credential issuance, verification, expiration, revocation, and versioning. DigitalCredential acts as a facade that simplifies application interaction while forwarding the original signer for authorization.

The application layer complements the contracts with managed institutional signing, PDF hashing, QR-based navigation, verification telemetry, event synchronization, analytics, and user interfaces.

A particularly important aspect of the project is that public verification does not require the verifier to use a wallet. The verifier interacts with an ordinary web interface while the application performs a read-only blockchain check.

The project also demonstrates clear boundaries. It is currently a local Hardhat prototype, authentication is simplified, development keys are unsuitable for production, SQLite has scalability limitations, and advanced decentralized identity technologies remain future work.

Within those limits, the implemented system provides a practical demonstration of how blockchain can be used as a credential trust and lifecycle layer without treating every part of the application as a blockchain problem.

---

36. References

1. National Institute of Standards and Technology, FIPS PUB 180-4: Secure Hash Standard (SHS), defining SHA-256 and related secure hash algorithms.

2. Ethereum Foundation, Ethereum Documentation, documentation covering Ethereum, smart contracts, accounts, transactions, and JSON-RPC concepts.

3. Solidity Project, Solidity Documentation, documentation for the Solidity smart-contract programming language.

4. OpenZeppelin, OpenZeppelin Contracts Documentation, particularly the "Ownable" access-control contract used by "InstitutionRegistry".

5. Nomic Foundation, Hardhat Documentation, development environment documentation for compiling, testing, and operating local Ethereum-compatible networks.

6. Ethers Project, Ethers.js v6 Documentation, covering JSON-RPC providers, wallets, contracts, events, and transaction interaction.

7. M. Jones, J. Bradley and N. Sakimura, RFC 7519 — JSON Web Token (JWT), Internet Engineering Task Force.

8. SQLite Project, SQLite Documentation, documentation for the embedded SQL database engine.

9. Node.js Project, Node.js Crypto API Documentation, including SHA-256 hash functionality used by the backend.

10. Mozilla Developer Network, Web Crypto API — SubtleCrypto.digest(), browser cryptography documentation relevant to the frontend SHA-256 implementation.

11. Express.js Project, Express Documentation, documentation for the Node.js web framework used by the REST API.

12. W3C, Verifiable Credentials Data Model, referenced only in relation to documented future interoperability work and not as an implemented CredChain feature.

---

37. Viva Voce Questions and Answers

Q1. Why did you use blockchain instead of only a traditional database?

A traditional database is still useful and CredChain uses SQLite for metadata and analytics. The blockchain is used only for the part where independent tamper evidence and cryptographically controlled state changes are valuable.

If someone changes the student name or status in SQLite, that does not change the SHA-256 digest stored in the blockchain credential record. Verification therefore obtains its trust result from the contract rather than treating the database as the final authority.

---

Q2. What exactly is stored on-chain and off-chain?

On-chain, CredChain stores the certificate ID, SHA-256 digest, issuer wallet, institution ID, issue timestamp, expiration timestamp, status, version number, version snapshots, and existence state.

Off-chain, it stores the student's full name, course name, application issue date, cached institution/status information, verification telemetry, and blockchain event cache.

Raw PDF files are processed in memory and are not intentionally persisted.

---

Q3. How does SHA-256 detect certificate tampering?

During issuance the complete PDF byte sequence is hashed using SHA-256.

During verification the up